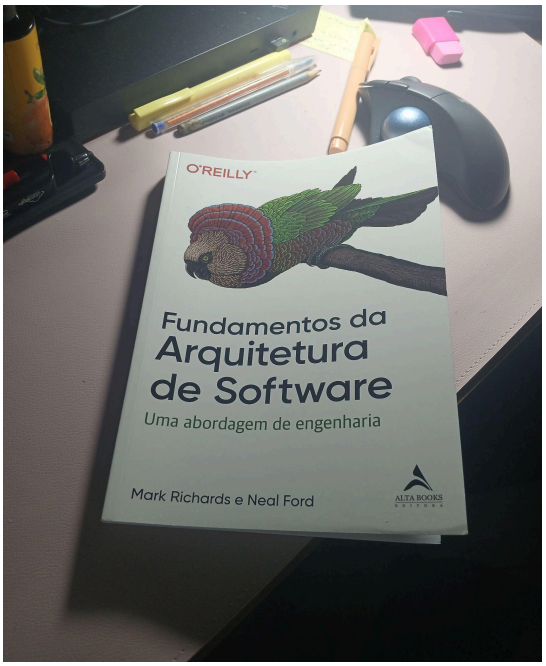
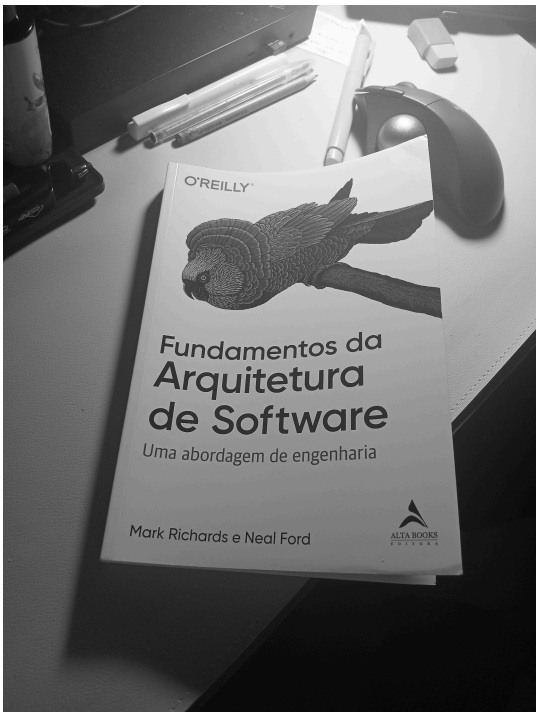
Estudo Dirigido 3 – Detecção de Bordas em Imagens

1. **Imagem original**



2. **Imagem em tons de cinza**



3. **Dimensões e características da imagem**

```python
# caracterização
altura, largura = I.shape
print(f"Altura: {altura} pixels")
print(f"Largura: {largura} pixels")
print(f"Total de pixels: {largura * altura}")
print(f"Intensidade mínima: {I.min()}")
print(f"Intensidade máxima: {I.max()}")
print(f"Intensidade média: {I.mean():.2f}")
```

```
Altura: 4080 pixels
Largura: 3060 pixels
Total de pixels: 12484800
Intensidade mínima: 1
Intensidade máxima: 255
Intensidade média: 111.05
```

Foi utilizada uma imagem própria obtida através da câmera do celular, de um livro com capa de alto contraste. A imagem foi carregada com o OpenCV e convertida para tons de cinza, de modo que cada pixel passou a ter uma única intensidade I(x, y) entre 0 e 255. Em seguida foram obtidas as dimensões, o número total de pixels e as intensidades mínima, máxima e média. (Como mostra os três tópicos mostrados acima)

4. Os Kernels Ku e Kv

```python
#  kernels de Sobel
Kv = np.array([[-1, 0, 1],
               [-2, 0, 2],
               [-1, 0, 1]])

Ku = Kv.T

print("\nKv = \n", Kv)
print("Ku = \n", Ku)
```

```
Kv =
 [[-1  0  1]
 [-2  0  2]
 [-1  0  1]]
Ku =
 [[-1 -2 -1]
 [ 0  0  0]
 [ 1  2  1]]
```

O kernel Kv foi definido explicitamente como uma matriz 3×3, com coeficientes que aproximam a derivada da intensidade em uma direção. O segundo kernel foi obtido pela transposta, $Ku = Kv^T$, que aproxima a derivada na direção ortogonal.

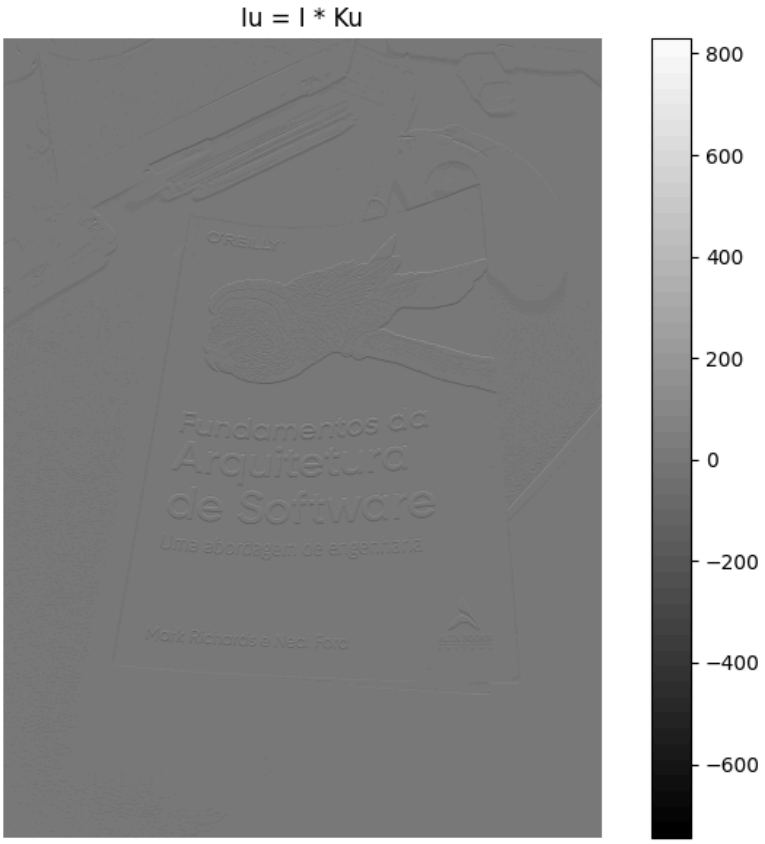
5. Convolução (código)

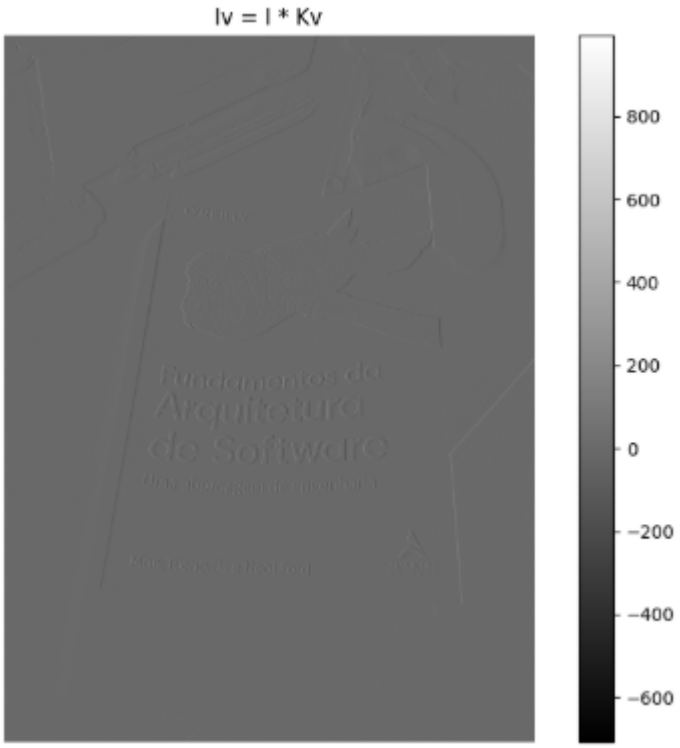
```
# convolução
If = I.astype(np.float64)
Iu = cv2.filter2D(If, cv2.CV_64F, np.flip(Ku))
Iv = cv2.filter2D(If, cv2.CV_64F, np.flip(Kv))
print("Iu:", Iu.dtype, Iu.min(), Iu.max())
print("Iv:", Iv.dtype, Iv.min(), Iv.max())
```

```
Iu: float64 -745.0 829.0
Iv: float64 -708.0 994.0
```

6. Imagem correspondente à componente Iu e Iv

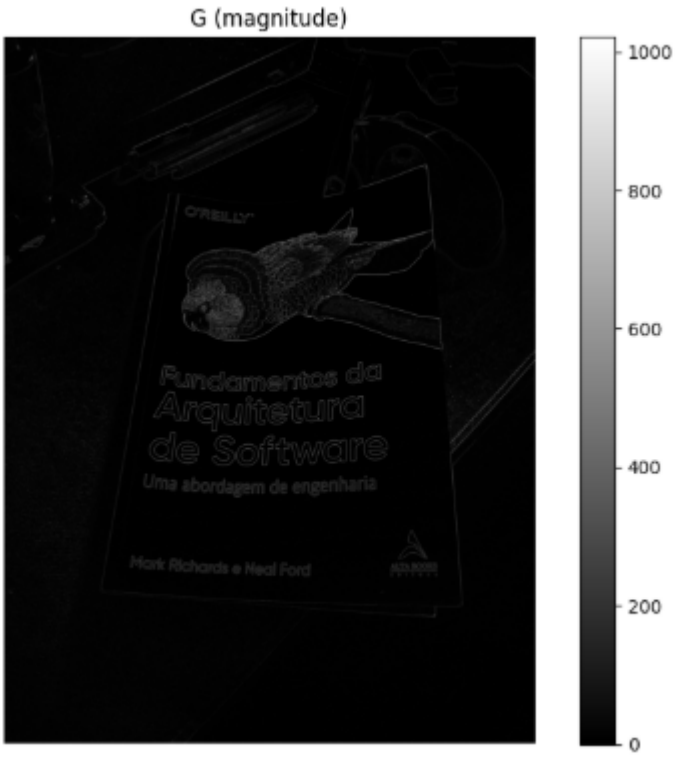
...



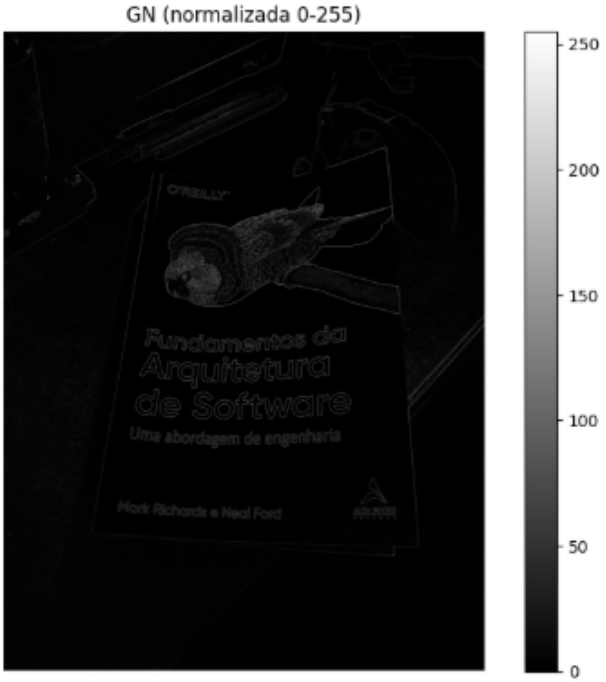


A imagem foi convolvida separadamente com cada kernel, gerando as componentes do gradiente Iu = I * Ku e Iv = I * Kv. A convolução foi feita em ponto flutuante (float64) para preservar os valores negativos, que indicam o sentido da transição (claro para escuro ou escuro para claro). Como a função `filter2D` do OpenCV calcula correlação, os kernels foram girados em 180° para obter a convolução verdadeira.

7. Imagem correspondente à magnitude do gradiente

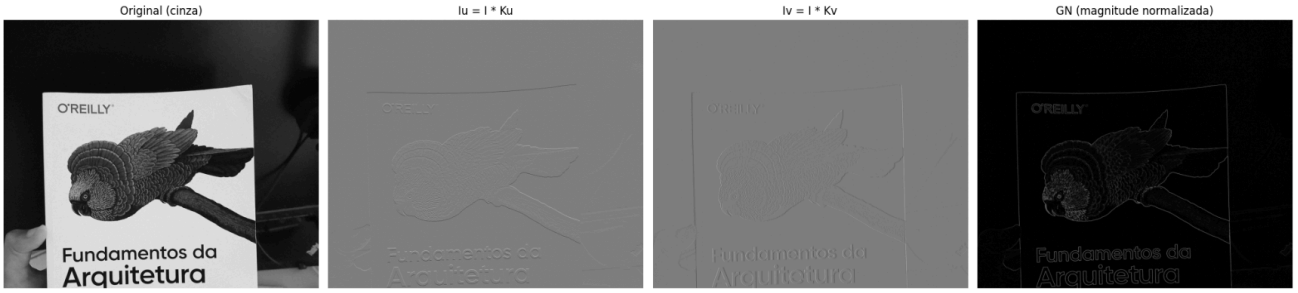


8. Magnitude normalizada do gradiente Gn



As duas componentes foram combinadas pixel a pixel por $G(x, y) = \sqrt{(Iu^2 + Iv^2)}$. Valores altos de G indicam variação forte de intensidade, ou seja, possíveis bordas. G foi então normalizada para a faixa [0, 255], resultando em Gn, o que facilita a visualização e a limiarização**.**

9. Comparação entre a imagem original, Ic, Iv e Gn



10. Valor do limiar manual Tg e imagem binária de bordas obtidas com o limiar manual

```
# limiar manual
TG = 50
EM = np.where(GN > TG, 255, 0).astype(np.uint8)

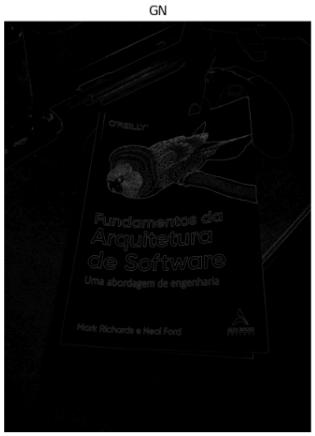
fig, ax = plt.subplots(1, 2, figsize=(14, 6))
ax[0].imshow(GN, cmap="gray"); ax[0].set_title("GN")
ax[1].imshow(EM, cmap="gray"); ax[1].set_title(f"Bordas (manual, TG = {TG})")
for a in ax: a.axis("off")
plt.tight_layout()
plt.show()
print("TG =", TG)
```




```
TG = 50
```

Foi escolhido um limiar TG a partir da observação de Gn e do seu histograma. Pixels com Gn > TG foram marcados como borda (255) e os demais como não borda (0), resultando na imagem binária E_M.

11. Valor do limiar automático, imagem binária de bordas obtida pelo método Otsu e comparação entre os resultados obtidos








```
TG = 50
TOtsu = 46.0
Pixels de borda (manual): 179851
Pixels de borda (Otsu):   196316
```

O método de Otsu calculou um limiar TOtsu a partir do histograma de Gn, escolhendo o valor que melhor separa os pixels em duas classes (maximiza a variância entre classes).